

TP2

CURACIÓN

Luego de habernos familiarizado con el dataset buscaremos responder las siguientes preguntas.

- ¿Cómo trataremos a los valores faltantes?
- ¿Cómo trataremos a los outliers?
- ¿Nos interesa todas las variables? ¿nos interesa agruparlas? ¿cómo?
- Si decidimos eliminar alguna, ¿en qué criterio nos basaremos?
- ¿Crearemos columnas nuevas?
- Si decidimos crear columnas nuevas, ¿usaremos una combinación de variables o anexaremos datos nuevos? ¿de dónde podríamos sacarlos?
- ¿Aplicaremos alguna transformación a nuestras variables? Normalizar, estandarizar, encoding, etc.
- ¿Cómo realizaremos la división de nuestro dataset? Antes o después de las transformaciones?
- Comencemos a pensar en el siguiente práctico, a grandes rasgos ¿Qué tipo de predicción podríamos hacer? ¿Qué modelos y métricas usaríamos?

Ahora que ya tenemos una idea de la calidad de nuestros datos es hora de tomar decisiones....

1. Eliminación de filas. Cuales? Duplicadas con faltantes
2. Eliminación de columnas. Cuales? Porque? Recomendación: aquellas que superan un 40% de faltantes y las que no aportan información al objetivo.
3. Imputar los valores faltantes de manera manual para los códigos que coinciden en municipio.
4. Definir variable target según objetivo (lo pueden redefinir)
5. Separar el dataset en dos/tres grupos: train, test, validación
6. Definir qué tipo de transformaciones vamos a utilizar según el tipo de dato y los modelos que vayamos a utilizar en el tp3.
7. Opcional: Utilizar pipeline

Eliminación Filas

```
[ ] _df_duplicados = agua_df[agua_df.duplicated(subset=['codigo','orden','sitios','fecha'],keep=False)]  
print(len(_df_duplicados))  
print(_df_duplicados[['codigo','orden','sitios','fecha']])
```



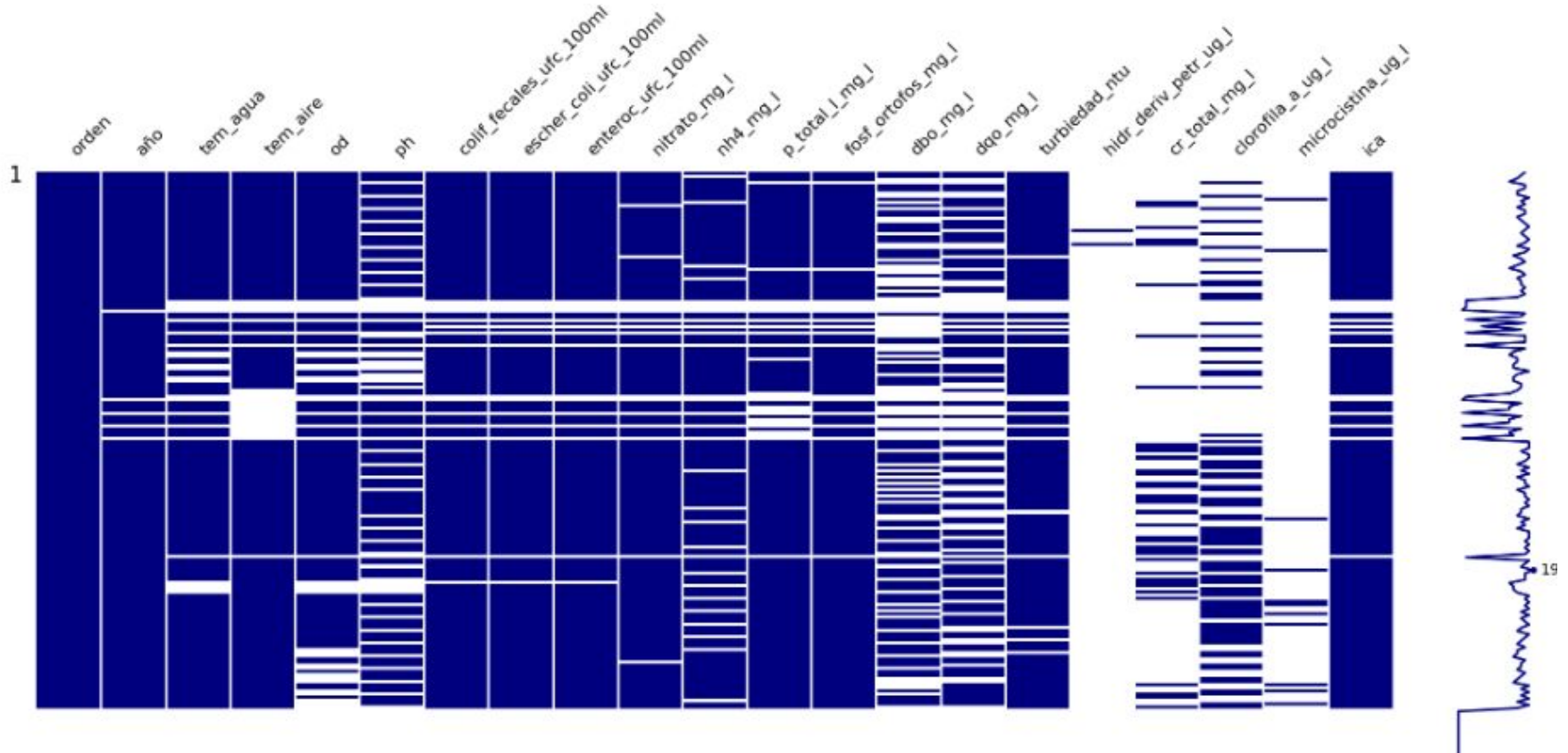
15

	codigo	orden	sitios	fecha
168	CA CU01	NaN	NaN	NaT
169	CA CU02	NaN	NaN	NaT
170	CA CU03	NaN	NaN	NaT
171	CA CU04	NaN	NaN	NaT
172	CA CU05	NaN	NaN	NaT
173	EN AD	NaN	NaN	NaT
177	EN AD	NaN	NaN	NaT
178	CA CU01	NaN	NaN	NaT
179	CA CU02	NaN	NaN	NaT
180	CA CU03	NaN	NaN	NaT
181	CA CU04	NaN	NaN	NaT
182	CA CU05	NaN	NaN	NaT
184	CA CU02	NaN	NaN	NaT
185	CA CU03	NaN	NaN	NaT
186	CA CU04	NaN	NaN	NaT

Eliminación columna

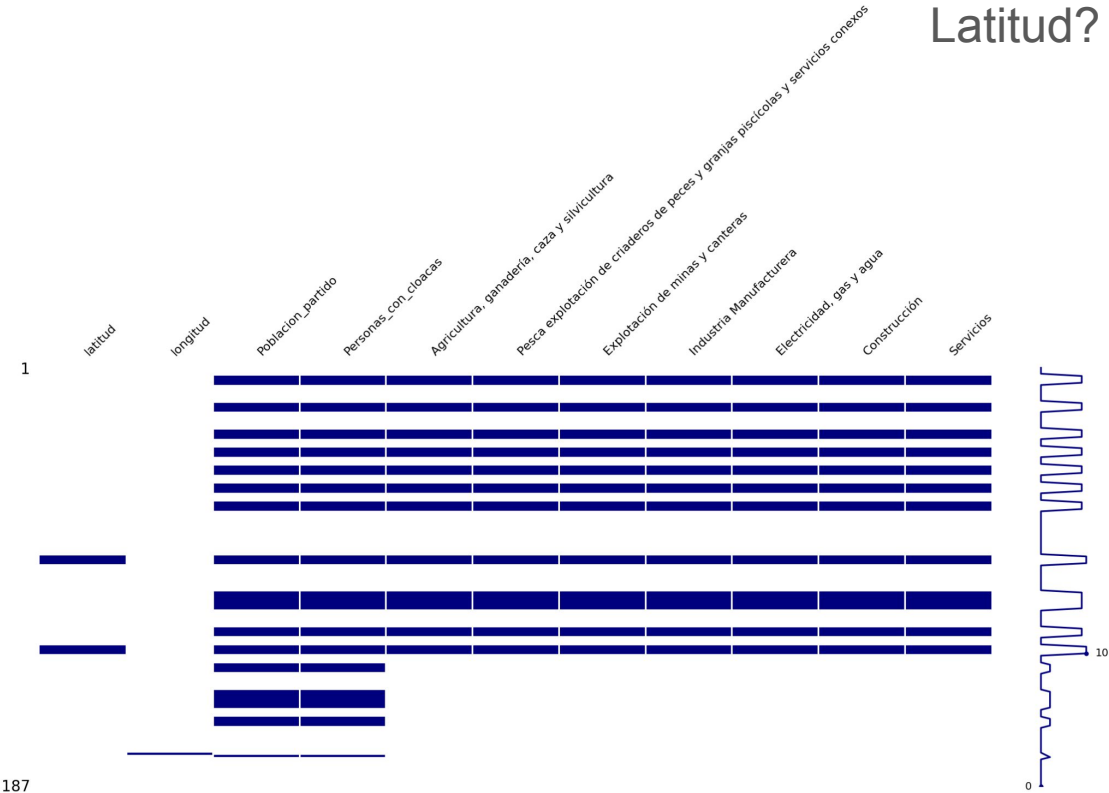
Orden? Sitios?

año? Fecha?



Eliminación columna

Actividad principal?
Longitud?
Latitud?



Imputación manual

Vemos que tenemos muchos faltantes, pero que sucede, la codificación nos puede ayudar a completar datos. Por ejemplo, los códigos que empiezan con TI son los de Tigre, podríamos programar para imputar los datos correspondientes en estas filas. Los datos que si tenemos están en https://github.com/MaricelSantos/Mentoria-Diplodatos-2024/blob/main/Dataset-crudos/agc_j_estaciones_monit_sust_qcas.csv

```
[ ] sitio_df["codigo"].unique()
```

```
⇒ array(['TI001', 'TI006', 'TI002', 'TI003', 'TI004', 'TI005', 'TI007',  
        'TI008', 'TI009', 'SF015', 'SI021', 'SI022', 'SI024', 'SI023',  
        'VL033', 'VL032', 'VL031', 'CA041', 'CA044', 'CA046', 'CA047',  
        'AV054', 'AV051', 'AV052', 'AV055', 'AV053', 'QU061', 'QU062',  
        'QU063', 'BZ078', 'BZ077', 'BZ080', 'EN-extra', 'EN081', 'EN082',  
        'EN083', 'EN084', 'BS092', 'BS095', 'BS091', 'BS094', 'BS093',  
        'CA CU01', 'CA CU02', 'CA CU03', 'CA CU04', 'CA CU05', 'EN AD',  
        'SF DELTA'], dtype=object)
```

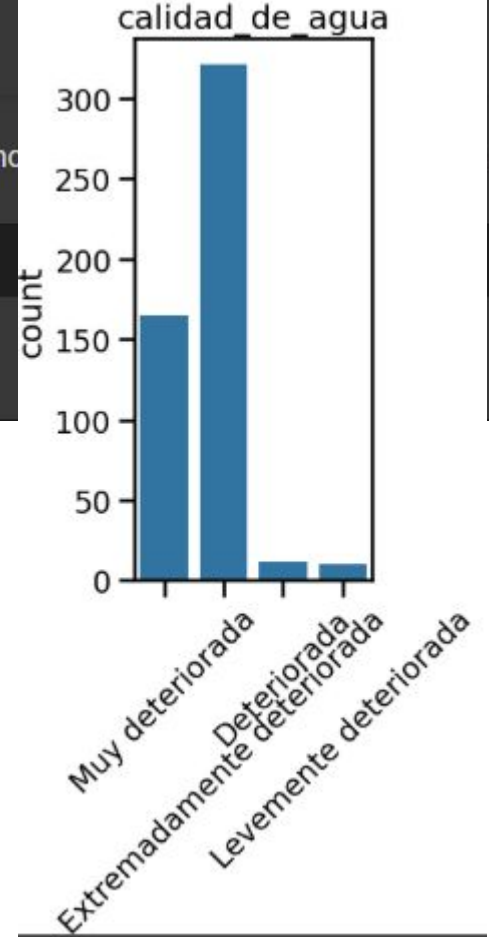
Variable Target/Objetivo

▼ Distribución de datos

Importante, tratar de ver que valores toma nuestra variable target. Lamentablemente no tenemos

```
[ ] main_df["calidad_de_agua"].unique()
```

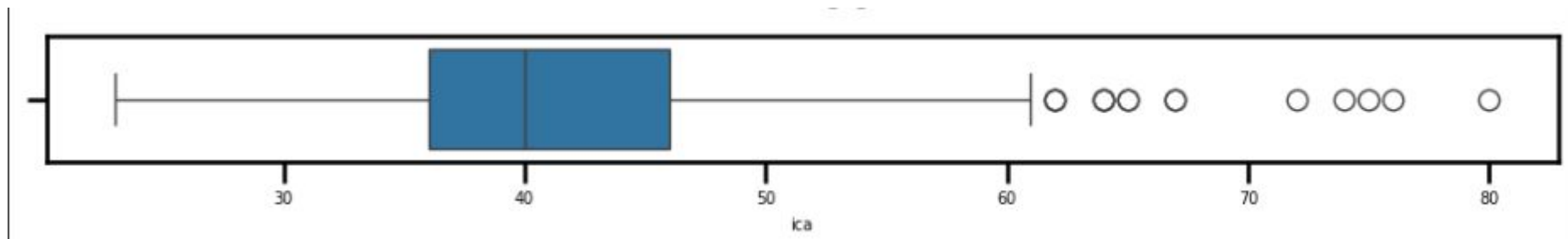
```
[↔] array(['Muy deteriorada', 'Extremadamente deteriorada', 'Deteriorada',  
        nan], dtype=object)
```



Aptitud para uso recreativo con ctto directo		Rango de valores	Descripción
APTA	Apta	95 - 100	La calidad del agua no presenta deterioro; las condiciones corresponden a los niveles deseados.
NO APTA	Levemente deteriorada	80 - 94	La calidad del agua presenta sólo un pequeño grado de deterioro; una muy baja proporción de parámetros (1 de 18) se alejan de lo deseado y este alejamiento es muy pequeño.
	Deteriorada	65 - 79	La calidad del agua se encuentra deteriorada debido a que algunos parámetros se alejan de las condiciones deseadas y este alejamiento no es muy significativo.
	Muy deteriorada	45 - 64	La calidad del agua está muy deteriorada; las condiciones de la varios parámetros están alejadas de los niveles esperados.
	Extremadamente deteriorada	0 - 44	La calidad del agua está extremadamente deteriorada y todos o varios de los parámetros se alejan ampliamente de los nivles esperados.

2021-2022-2023

521 Entradas



SEPARACIÓN TRAIN - TEST - VALIDACIÓN

Train: datos entrenamiento (70%)

Test: datos para ajustar hiperparametros (20%)

Validación: datos para probar el modelo final (10%)

Opción: usar datos 2024 cómo validación. Sugerencia, mirar con lupa estos datos para ver si la decisión tiene sentido.

Transformaciones

dependiendo el tipo de variable es que realizaremos distintas transformaciones

Tipo de datos

Datos categóricos/binarios: Presencia, Ausencia

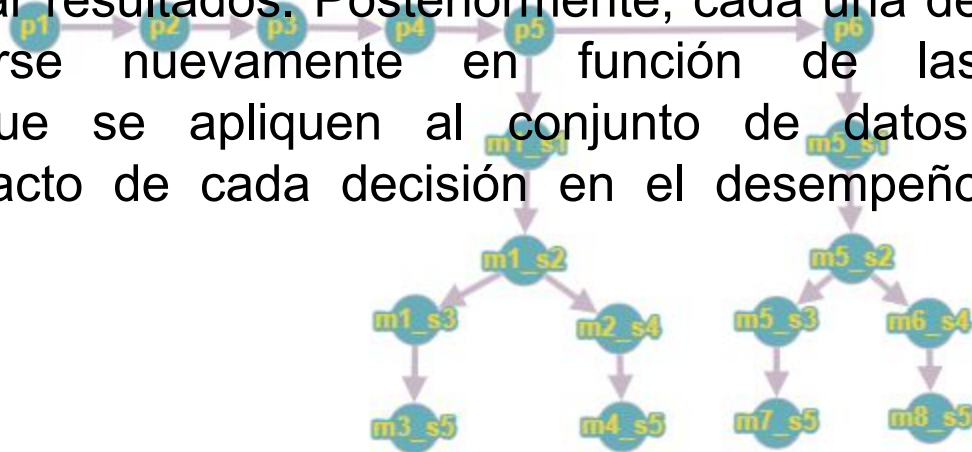
Datos categóricos nominales: Campaña, calidad de agua, gobierno local, actividad principal

Datos ordinales: Actividades de municipios.

Datos numéricos continuos: ph, tem_agua, tem_aire, od, dbo_mg_l, dco_mg_l, nitrato_mg_l, nh4_mg_l, fosf_ortofos_mg_l, p_total_l_mg_l, turbiedad_ntu, hidr_deriv_petr_ug_l, cd_total_mg_l, cr_total_mg_l, clorofila_a_ug_l y microcistina_ug_l, ica.

Transformaciones - Presentar al menos 2 dataset para comparar.

La propuesta es que trabajen de forma análoga al diagrama que se presenta, donde los primeros pasos contemplan la limpieza inicial del conjunto de datos (eliminación de columnas y filas). Luego el flujo puede ramificarse según las decisiones que se deseen explorar. Por ejemplo, una posible línea de trabajo podría considerar la eliminación de variables altamente correlacionadas, mientras que otra podría mantener todas las variables originales para comparar resultados. Posteriormente, cada una de estas ramas puede ramificarse nuevamente en función de las transformaciones adicionales que se apliquen al conjunto de datos, permitiendo así analizar el impacto de cada decisión en el desempeño del/los modelo/s.



IMPUTACIÓN

- KNN
- Bayesiana
- Hot-deck

Formas de comparar cual es la mejor opción:

Ver cual copia mejor la distribución del dataset original

Aspecto	KNN Imputation	Bayesiana
Tipo de imputación	Basada en distancia (vecindad)	Basada en modelo probabilístico
Tipo de datos	Numéricos y categóricos	Numéricos principalmente
Uso de correlaciones	✅ Sí (implícitamente)	✅ Sí (modelo explícito)
Consistencia estadística	Variable según distribución	Alta, especialmente con múltiples imputaciones
Ventaja principal	Fácil, capta relaciones locales	Rigor estadístico, captura incertidumbre
Desventaja principal	Sensible a escala y ruido	Complejo de implementar y afinar
Apto para datos faltantes MAR (Missing At Random)	✅ Sí	✅ Sí
Apto para MNAR (Missing Not At Random)	❌ No	🟡 A veces (depende del modelo)

Escalado: Centramos en cero y con desviación estándar de 1, ajustamos todas las variables a la misma escala. Asumimos distribución normal para las variables.

Normalización: Llevar todo a una escala de 0 a 1. Asumimos que la distribución de los datos no es importante.

Cuando usar cada uno? Depende del algoritmo a utilizar. Algunos asumen la distribución normal de los datos, otros no necesitan hacerlo.
Nuestros datos presentan muchos outliers? Outliers evidentes?

Modelo	¿Escalar/Normalizar?
k-NN, SVM, regresión, redes neuronales	✅ Sí
Árboles, Random Forest, XGBoost	❌ No
Rankings con muy pocos valores únicos	🧐 Depende, pero probablemente no

Cuando usar cada uno? Depende del algoritmo a utilizar. Algunos asumen la distribución normal de los datos, otros no necesitan hacerlo. Algunos son sensibles a las escalas y es necesario normalizar.

Nuestros datos presentan muchos outliers? Outliers evidentes?

OneHotEncoding: Crea una columna binaria para cada categoría. Incrementa la dimensionalidad. No se usa en ordinales

LabelEncoding: Asignan un valor a cada categoría. Útil en modelos que pueden recibir números enteros sin asumir un orden (Árboles de decisión)

Quando usar cada uno? Depende del algoritmo a utilizar.

	One-Hot Encoding	Label Encoding
Tipo de variable	Categórica nominal (sin orden)	Categórica nominal o ordinal
Representación	Binaria (0/1 por categoría)	Enteros consecutivos
Adecuado para modelos lineales	✅ Sí. Evita suposiciones de orden	❌ No. Puede inducir relaciones ficticias
Árboles de decisión / Random Forest	❌ Puede aumentar dimensionalidad innecesariamente	✅ Sí. Árboles pueden manejar enteros sin problema
Modelos basados en distancia (KNN, SVM)	✅ Sí. Más preciso para variables categóricas	❌ No. Distancias pueden ser engañosas
Ventajas	No induce orden, interpreta bien categorías	Compacto, simple, rápido
Desventajas	Alta dimensionalidad con muchas categorías	Asume orden numérico donde puede no haberlo

Opcional: utilizar pipelines

Les dejo el enlace a un colab donde les muestro rápidamente como funcionaria y la comparación con no utilizarlos....

<https://colab.research.google.com/drive/1ONesn5eNEvA5xFu4oWbsAOQ4Zr5kITbl#scrollTo=4WBXpERozJc7>

✓ Definición de Columnas y Transformaciones

Es necesario identificar las columnas numéricas y categóricas porque vamos a tener diferentes transformaciones según cada tipo de columna.

numeric_transformer contiene un pipeline con dos pasos, imputar que utiliza el metodo SimpleImputer con la mediana como argumento y escalar que utiliza un estandar escalar. categorical_transformer hace lo propio con las categoricas y luego preprocessor agrupa los transformadores para las columnas. Hasta aca tenemos de forma ordenada los procedimientos para cada columna.

```
[ ] numeric_features = ['num_col1', 'num_col2']
    categorical_features = ['cat_col1', 'cat_col2']

    numeric_transformer = Pipeline(steps=[
        ('imputar', SimpleImputer(strategy='median')),
        ('scalar', StandardScaler())])

    categorical_transformer = Pipeline(steps=[
        ('imputar', SimpleImputer(strategy='constant', fill_value='missing')),
        ('onehot', OneHotEncoder(handle_unknown='ignore'))])

    preprocessor = ColumnTransformer(
        transformers=[
            ('num', numeric_transformer, numeric_features),
            ('cat', categorical_transformer, categorical_features)])
```

Esto es parte del TP3 pero se los muestro para que se entienda como se usa el pipeline. En este caso el ejemplo es con un Random Forest Regressor

```
[ ] # Pipeline que incluye el preprocesamiento y el modelo RandomForestRegressor
pipeline = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('regressor', RandomForestRegressor(n_estimators=100, random_state=42))])
```

✓ Utilización del pipeline

```
[ ]
#Entrenamiento

pipeline.fit(x_train, y_train)

#Predicción

y_pred = pipeline.predict(x_test)

#Metricas

mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print(f'Mean Squared Error: {mse}')
print(f'R^2 Score: {r2}')
```

Importante!

Pueden trabajar utilizando *pipelines* para organizar y encadenar los distintos pasos del preprocesamiento y modelado. No obstante, independientemente de si utilizan *pipelines* o aplican los transformadores por separado, es fundamental respetar una buena práctica clave: cada transformador debe ser ajustado (`fit_transform`) únicamente sobre los datos de entrenamiento. Posteriormente, se debe aplicar el método `transform` sobre los datos de prueba, utilizando los parámetros aprendidos del conjunto de entrenamiento. Esto garantiza que no se filtre información del conjunto de prueba durante el proceso de entrenamiento y se mantenga la validez del modelo.